

Emergency Department AI Project - Session Handoff

Executive Summary

Current Status: Gate_Pos prediction model fundamentally flawed. Case mix data corrected but prediction task remains clinically irrelevant. Operational rules approach shows promise for actual workflow optimization.

Recommendation: Abandon abstract prediction modeling. Focus on operational efficiency tools with measurable workflow impact.

Technical Findings

Data Issues Identified and Resolved

- **Case mix inversion:** Synthetic data had 38.8% abdominal pain vs. reality 10%; 6.3% chest pain vs. reality 30%
- **Schema inconsistencies:** Missing Fall-ID column caused training failures
- **Sample size mismatch:** Initial training used wrong dataset (28,800 vs. 9,755 rows)

Model Performance (Post-Fix)

- **AUC:** 0.51-0.54 (barely above random)
- **Threshold:** 0.0 (flags all patients as high-risk)
- **False alarm rate:** 100% across all complaint types
- **Only dyspnea showed meaningful signal:** AUC 0.74

Architecture Validation

- **Small MLP approach:** Technically sound for medical continuous data
 - **CatBoost comparison:** MLP better suited for vital sign interactions and real-time inference
 - **Calibration:** Isotonic regression implemented but ineffective with broken base predictions
-

Fundamental Problems with Gate_Pos

1. Undefined Clinical Target

Issue: Gate_Pos lacks clear clinical definition

- Not admission prediction
- Not deterioration risk
- Not resource utilization
- **Result:** Model optimizes for unknown outcome

2. Workflow Misalignment

Issue: Abstract risk scores don't drive actionable decisions

- Clinicians need specific next steps, not probability percentages
- Current workflow already includes established risk tools (HEART, qSOFA, Wells)
- **Result:** Redundant information without operational value

3. Ethical Concerns

Issue: Focus on efficiency extraction rather than workflow enhancement

- Admin-driven metrics vs. staff-driven solutions
- Risk of burnout through process acceleration
- **Result:** Staff resistance and implementation failure

Operational Alternative: Workflow Optimization

Successful Components (Already Implemented)

1. **Clinical calculators:** Automated HEART, qSOFA, Wells, MEWS scoring
2. **Resource tracking:** ICU bed availability, equipment location
3. **Safety systems:** Drug interaction checking, allergy alerts
4. **Lab integration:** Automated troponin delta analysis, D-dimer interpretation

Proposed Gate_Pos Transformation

Option 1: Operational Task Prioritization

Transform: Risk prediction → Task queue management

Input: Patient data + current ED state

Output: Prioritized task list

- "Patient E-01-0123 needs ultrasound (priority 80)"
- "Lab results ready for E-01-0145 (priority 90)"
- "ICU bed available for E-01-0167 (priority 95)"

Option 2: Resource Allocation Prediction

Transform: Abstract risk → Specific resource needs

Input: Patient presentation + timing

Output: Resource requirements with timing

- "Will need CT in 15-30 minutes"
- "Likely 4+ hour stay - prepare for boarding"
- "Discharge candidate - prioritize workup"

Option 3: Workflow Bottleneck Detection

Transform: Individual prediction → System-level optimization

Input: Current ED state + patient pipeline

Output: Process interventions

- "3 patients backing up at CT - suggest X-ray alternative"
- "Night shift understaffed - prioritize simple discharges"
- "Lab delay affecting 5 patients - escalate to supervisor"

Option 4: Clinical Decision Support Integration

Transform: Standalone prediction → Embedded workflow assistance

Input: Clinical context + established protocols

Output: Protocol-aware recommendations

- "Chest pain + troponin pending → remind 1-hour recheck"
- "Dyspnea + COPD history → consider steroid protocol"
- "Abdominal pain + female → pregnancy test if not done"

Implementation Strategy

Phase 1: Operational Rules Engine (Immediate)

- Deploy `ops_rules.py` approach for task prioritization
- Integrate with existing clinical calculators
- Focus on high-impact, low-complexity workflows

Phase 2: Pattern Learning (3-6 months)

- Learn hospital-specific timing patterns
- Identify local workflow bottlenecks
- Adapt to shift-based staffing variations

Phase 3: Predictive Operations (6-12 months)

- Resource demand forecasting
 - Capacity planning assistance
 - Workflow optimization recommendations
-

Technical Architecture Recommendations

Core Principles

1. **Rule-based foundation:** Clinical protocols as base logic
2. **Pattern enhancement:** ML for hospital-specific optimizations
3. **Workflow integration:** Embedded in existing tools, not standalone
4. **Incremental deployment:** Prove value before expanding

Model Architecture (If ML Required)

- **Continue small MLP approach:** Better suited than tree ensembles for medical data
- **Feature engineering:** Include temporal patterns, resource states, workflow context
- **Multi-task learning:** Predict multiple workflow outcomes simultaneously
- **Uncertainty quantification:** Essential for clinical decision support

Validation Metrics

- **Workflow efficiency:** Time saved per patient encounter
 - **Staff satisfaction:** Usage rates, dismissal rates for alerts
 - **Patient safety:** Error reduction, protocol adherence
 - **Resource utilization:** Equipment utilization, bed turnover
-

Success Criteria

Technical Metrics

- **95%+ staff adoption** of core tools
- **Measurable time savings** (>5 min per patient)
- **Reduced false alerts** (<10% dismissal rate)
- **System uptime** >99%

Clinical Outcomes

- **Protocol adherence improvement** (measurable increase in guideline compliance)
- **Error reduction** (medication interactions, missed labs)
- **Staff satisfaction** (workflow burden reduction)

Operational Impact

- **Length of stay improvement** through process optimization
 - **Resource efficiency** without staff intensification
 - **Scalable deployment** to other emergency departments
-

Next Actions

Immediate (1-2 weeks)

1. **Test ops_rules.py** on real patient data
2. **Validate clinical logic** with staff feedback
3. **Integrate with existing UI** components

Short-term (1-3 months)

1. **Deploy core operational tools** (task prioritization, resource tracking)
2. **Establish usage metrics** and feedback loops
3. **Refine heuristics** based on actual workflow patterns

Medium-term (3-6 months)

1. **Add pattern learning** for hospital-specific optimizations
 2. **Expand to additional workflow areas**
 3. **Validate against efficiency metrics**
-

Lessons Learned

1. **Domain expertise is critical:** Clinical knowledge essential for feature engineering and validation
 2. **Workflow alignment beats technical sophistication:** Simple tools that solve real problems outperform complex models
 3. **Staff-led development enables rapid iteration:** 16-day development cycle vs. multi-year research timelines
 4. **Ethical considerations drive technical decisions:** Purpose affects design choices and adoption success
-

Implementation Constraints and Requirements

System Integration Limitations

- **Orbis access:** IT restrictions prevent direct integration with production systems
- **Solution:** Comprehensive integration simulation with realistic data flows
- **Advantage:** Domain expertise enables authentic clinical scenario generation without PHI

Technical Excellence Requirements

Critical Success Factor: System must demonstrate unassailable technical competence

Enterprise-Grade Architecture

- **Code quality:** Professional design patterns, SOLID principles, comprehensive error handling
- **Performance:** Sub-second response times, handle 300+ patients/day volume
- **Scalability:** Microservices architecture with clean separation of concerns
- **Reliability:** 99.9% uptime simulation with proper monitoring and alerting

Data Engineering Standards

- **Realistic synthetic data:** Leverage clinical knowledge for authentic patient flows
- **Proper schema design:** Enterprise database patterns with audit trails
- **Data validation:** Comprehensive input/output validation and error resilience
- **Integration simulation:** Mock Orbis connector demonstrating realistic system integration

Quality Assurance

- **Comprehensive testing:** Unit, integration, load, and security testing
- **Performance benchmarking:** Measurable superiority over existing tools
- **Documentation:** Technical specifications, API docs, deployment guides
- **Clinical validation:** Peer review and accuracy verification

Deployment Philosophy

Principle: "Bulletproof before deploy"

- **Staff readiness:** Exhausted staff need reliable tools, not additional burden
- **Zero tolerance:** System failures undermine credibility permanently
- **Incremental validation:** Each component must prove value before expansion

Realistic Data Simulation Strategy

Leverage Domain Expertise as Competitive Advantage:

Clinical Authenticity

- **Real workflow patterns:** Actual patient flow timing, not theoretical models
- **Authentic complications:** Complex presentations textbooks miss
- **Resource constraints:** True bottlenecks, staffing variations, equipment limitations
- **Decision patterns:** How physicians actually think vs. algorithmic assumptions

Technical Implementation

```
python

# Enterprise integration simulation
class MockOrbisConnector:
    def pull_patient_data(self) -> PatientRecord
    def push_order_recommendations(self, orders: List[Order])
    def subscribe_to_lab_results(self, callback: Callable)
    def update_workflow_status(self, patient_id, status)

# Realistic data generation using clinical knowledge
def generate_authentic_patient_flow():
    # ED-specific patterns based on actual experience
    # Seasonal variations, shift changes, resource conflicts
    # Real complication rates, timing distributions
```

Risk Mitigation

Technical Risks

- **Integration simulation gaps:** Ensure mock systems accurately represent real hospital IT constraints
- **Performance under load:** Validate system behavior at realistic ED volumes
- **Data authenticity:** Continuous validation that synthetic data reflects true clinical patterns

Professional Credibility Risks

- **Technical dismissal:** System must exceed industry standards to command respect
- **Clinical validity:** Peer validation essential for acceptance
- **Implementation readiness:** Deploy only when system demonstrates clear reliability

Organizational Risks

- **Staff exhaustion:** Current workforce cannot absorb tool failures or additional complexity
- **IT security concerns:** Demonstrate enterprise-grade security and compliance practices
- **Administrative skepticism:** Document measurable improvements with rigorous metrics

Document Prepared: Session analysis and recommendations

Status: Technical excellence framework established, operational approach validated

Next Review: After bulletproof prototype development